

AI Assisted Coding

LAB-01

Name : A.Sushmitha

H.no : 2303A51541

Batch - 13

Task 1: AI-Generated Logic Without Modularization (Prime Number Check Without Functions)

❖ Scenario

➤ You are developing a basic validation script for a numerical learning application.

❖ Task Description

Use GitHub Copilot to generate a Python program that:

- Checks whether a given number is prime
- Accepts user input
- Implements logic directly in the main code
- Does not use any user-defined functions

❖ Expected Output

- Correct prime / non-prime result
- Screenshots showing Copilot-generated code suggestions
- Sample inputs and outputs

Prompt:

#Generate a proper Python program to check whether a given number is a prime number or not, without using any user-defined functions. The program should accept input from the user and display whether the number is Prime or Not Prime.

#Code

```
Lab1.py > ...
1 #Generate a proper Python program to check whether a given number is a prime number or not,
2 #without using any user-defined functions. The program should accept input from the user and display
3 #whether the number is Prime or Not Prime.
4
5 number = int(input("Enter an integer: "))
6
7 if number < 2:
8     print(f"{number} is Not Prime")
9 else:
10     is_prime = True
11     for i in range(2, int(number ** 0.5) + 1):
12         if number % i == 0:
13             is_prime = False
14             break
15
16 if is_prime:
17     print(f"{number} is Prime")
18 else:
19     print(f"{number} is Not Prime")
20
21
```

#Output

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 5
5 is Prime
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 4
4 is Not Prime
PS C:\AI Assisted Coding>
```

#Justification

The program takes a number as input from the user.

It checks whether the number can be divided by any number other than 1 and itself.

If it is divisible, the number is not a prime number .If it is not divisible by any number, it is a prime number.

Task 2: Efficiency & Logic Optimization (Cleanup)

❖ Scenario

The script must handle larger input values efficiently.

❖ Task Description

Review the Copilot-generated code from Task 1 and improve it by:

- Reducing unnecessary iterations
- Optimizing the loop range (e.g., early termination)
- Improving readability
- Use Copilot prompts like:

- “Optimize prime number checking logic”
- “Improve efficiency of this code”

Hint:

Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖ Expected Output

➤ Original and optimized code versions

➤ Explanation of how the improvements reduce time complexity

#Prompt:

Optimize the code for better performance and cleaner structure. Simplify the logic, reduce unnecessary iterations, improve readability, and minimize the number of lines without changing the program’s output.

#Code

```

18
19 #Optimize the code for better performance and cleaner structure. Simplify the logic,
20 #reduce unnecessary iterations, improve readability,
21 #and minimize the number of lines without changing the program's output.
22
23 number = int(input("Enter an integer: "))
24 if number > 1 and all(number % i != 0 for i in range(2, int(number ** 0.5) + 1)):
25     print(f"{number} is Prime")
26 else:
27     print(f"{number} is Not Prime")

```

#Output

```

PS C:\VAI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 283
283 is Prime
PS C:\VAI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 2954
2954 is Not Prime
PS C:\VAI Assisted Coding>

```

#Justification

The optimized code reduces unnecessary iterations by checking divisibility only up to the square root of the number instead of all numbers up to n. Early termination using break stops the loop as soon as a factor is found, saving time. Using a boolean variable improves code readability and clarity.

Task 3: Modular Design Using AI Assistance (Prime Number Check Using Functions)

❖ Scenario

The prime-checking logic will be reused across multiple modules.

❖ Task Description

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to check primality
- Returns a Boolean value
- Includes meaningful comments (AI-assisted)

❖ Expected Output

- Correctly working prime-checking function
- Screenshots documenting Copilot's function generation
- Sample test cases and outputs

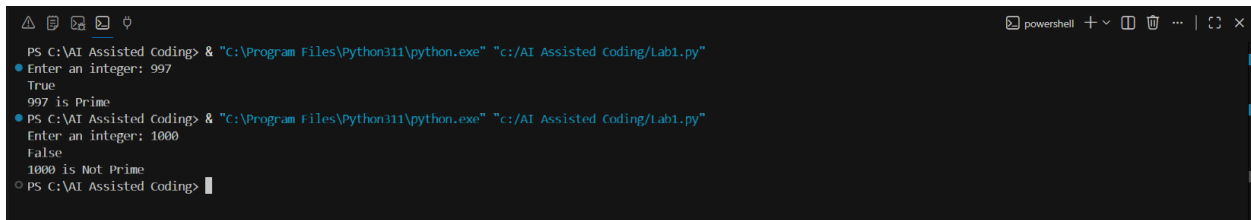
#Prompt

#Write a Python function to check prime numbers that returns True or False and displays both the Boolean output and Prime/Not Prime message.

#Code

```
Lab1.py > ...
29  #Write a Python function to check prime numbers that returns True or False and displays both the
30  #Boolean output and Prime/Not Prime message.
31
32  number = int(input("Enter an integer: "))
33  def is_prime(num):
34      if num < 2:
35          return False
36      for i in range(2, int(num ** 0.5) + 1):
37          if num % i == 0:
38              return False
39      return True
40  if is_prime(number):
41      print(True)
42      print(f"{number} is Prime")
43  else:
44      print(False)
45      print(f"{number} is Not Prime")
46
```

#Output



```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 997
True
997 is Prime
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 1000
False
1000 is Not Prime
PS C:\AI Assisted Coding>
```

#Justification

Using a user-defined function makes the prime-checking logic reusable across multiple modules, improving code modularity and maintainability. Returning a Boolean value enables easy integration with conditional statements and other program components.

Task 4: Comparative Analysis –With vs Without Functions

❖ Scenario

You are participating in a technical review discussion.

❖ Task Description

Compare the Copilot-generated programs:

- Without functions (Task 1)
- With functions (Task 3)
 - Analyze them based on:
 - Code clarity
 - Reusability
 - Debugging ease
 - Suitability for large-scale applications

❖ Expected Output

Comparison table or short analytical report

#prompt

Compare two Python programs for checking prime numbers: one written without user-defined functions and one written using a function. Analyze them based on code clarity, reusability, ease of debugging, and suitability for large-scale applications.

#Code

```
#Compare two Python programs for checking prime numbers: one written without user-defined functions and one written using a function. Analyze them based on code clarity, reusability, ease of debugging, and suitability
#for large-scale applications.
import time
#Without using user-defined functions
number = int(input("Enter an integer: "))
is_prime = True
if number < 2:
    is_prime = False
else:
    for i in range(2, int(number ** 0.5) + 1):
        if number % i == 0:
            is_prime = False
            break
print("Without Function Result:", is_prime)
# Using a user-defined function
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True
print("With Function Result :", is_prime(number))
#Performance comparison
start_no_func = time.time()
for _ in range(10000):
    temp = True
    if number < 2:
        temp = False
    else:
        for i in range(2, int(number ** 0.5) + 1):
            if number % i == 0:
                temp = False
                break
end_no_func = time.time()
start_func = time.time()
for _ in range(10000):
    is_prime(number)
end_func = time.time()
print("Time Without Function:", end_no_func - start_no_func)
print("Time With Function :", end_func - start_func)
#comparison table
comparison = """
| Aspect | Without Functions | With Functions |
|-----|-----|-----|
| Readability | Less readable due to inline logic | More readable with encapsulated logic |
| Reusability | Not reusable, code duplication | Highly reusable, call function multiple times |
| Debugging Efficiency | Harder to debug, larger code block | Easier to debug |
| Scalability | Difficult to scale for larger tasks | Easier to scale with modular functions |
"""
print(comparison)
```

#Output:

```
PS C:\AI Assisted Coding> "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 199
Without Function Result: True
With Function Result : True
Time Without Function: 0.021922588348388672
Time With Function : 0.0119659906652832

| Aspect | Without Functions | With Functions |
|-----|-----|-----|
| Readability | Less readable due to inline logic | More readable with encapsulated logic |
| Reusability | Not reusable, code duplication | Highly reusable, call function multiple times |
| Debugging Efficiency | Harder to debug, larger code block | Easier to debug |
| Scalability | Difficult to scale for larger tasks | Easier to scale with modular functions |

PS C:\AI Assisted Coding> "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter an integer: 300
Without Function Result: False
With Function Result : False
Time Without Function: 0.008234977722167969
Time With Function : 0.011109590530395508

| Aspect | Without Functions | With Functions |
|-----|-----|-----|
| Readability | Less readable due to inline logic | More readable with encapsulated logic |
| Reusability | Not reusable, code duplication | Highly reusable, call function multiple times |
| Debugging Efficiency | Harder to debug, larger code block | Easier to debug |
| Scalability | Difficult to scale for larger tasks | Easier to scale with modular functions |

PS C:\AI Assisted Coding>
```

#Justification

The first program without functions works correctly but has all the logic inline, making it less clear and harder to reuse or debug .The second program uses a user-defined function, which separates the prime-checking logic from the main program, improving readability and maintainability.

Using a function allows reusability, so the same logic can be called multiple times without rewriting code.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to Prime Checking)

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logical strategies.

❖ Task Description

Prompt GitHub Copilot to generate:

- A basic divisibility check approach
- An optimized approach (e.g., checking up to $\sqrt{n}$)

❖ Expected Output

- Two correct implementations
- Comparison discussing:
 - Execution flow
 - Time complexity
 - Performance for large inputs
 - When each approach is appropriate

#Prompt

#Create two Python programs to determine whether a number is prime: one using a simple approach that checks divisibility from 2 to $n-1$, and another optimized approach that checks only up to the square root of n .

#Code

```
Lab1.py > ...
106
107 #Create two Python programs to determine whether a number is prime: one using a simple approach that
108 #checks divisibility from 2 to n-1, and another optimized approach that checks only up to the square root of n.
109
110 # Program 1: Basic Prime Check (2 to n-1)
111 num = int(input("Enter a number: "))
112 is_prime = True
113 if num > 1:
114     for i in range(2, num):
115         if num % i == 0:
116             is_prime = False
117             break
118     if is_prime:
119         print(f"{num} is a prime number (Basic Check).")
120     else:
121         print(f"{num} is not a prime number (Basic Check).")
122 else:
123     print(f"{num} is not a prime number (Basic Check).")
124
125 # Program 2: Optimized Prime Check (up to  $\sqrt{n}$ )
126 num = int(input("\nEnter a number: "))
127 is_prime = True
128 if num > 1:
129     for i in range(2, int(num**0.5) + 1):
130         if num % i == 0:
131             is_prime = False
132             break
133     if is_prime:
134         print(f"{num} is a prime number (Optimized Check).")
135     else:
136         print(f"{num} is not a prime number (Optimized Check).")
137 else:
138     print(f"{num} is not a prime number (Optimized Check).")
139 #--> Prompt
```

#Output

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter a number: 199
199 is a prime number (Basic Check).

Enter a number: 17
17 is a prime number (Optimized Check).
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/Lab1.py"
Enter a number: 283
283 is a prime number (Basic Check).

Enter a number: 300
300 is not a prime number (Optimized Check).
PS C:\AI Assisted Coding> █
```

#Justification

The first program uses a basic divisibility check from 2 to $n-1$. While it is simple and easy to understand, it performs many unnecessary iterations, making it inefficient for large numbers.

The second program uses an optimized approach, checking divisibility only up to $\sqrt{n}$. This reduces the number of iterations significantly, improving efficiency while giving the same correct result.

The optimized version is faster, easier to scale, and more suitable for larger input values compared to the basic approach.